

Drone-Based Delivery Optimization Problem

Mathematical Modeling, Complexity, and Algorithmic Solutions

University Group Project

June 9, 2026

Objective: Route a homogeneous fleet of drones to deliver payloads from a central depot to distributed customers.

Physical & Operational Constraints:

- **3D Environment:** Navigation occurs in a three-dimensional airspace.
- **No-Fly Zones (NFZs):** Drones must avoid restricted static obstacles.
- **Capacity Limits:** Strict bounds on **payload** weight and **battery** energy.
- **Anisotropic Energy Profile:** Energy consumption rates vary significantly between horizontal flight and vertical ascent/descent.

How the Data was Generated:

- **Grid Construction:** A simulated three-dimensional airspace grid where customers are distributed at varying altitudes.
- **No-Fly Zones (NFZs):** Modeled as 3D bounding boxes restricting flight paths (e.g., restricted airspace, tall buildings).
- **Drone Specifications:** Configured with maximum payload capacities and anisotropic battery depletion rates.
- **Export:** The entire environment (depot, customers, demands, drones, and NFZs) is exported as reproducible JSON benchmark instances.

Challenge: Embedding full 3D obstacle avoidance within routing formulations is computationally intractable.

Solution: A* Pathfinding Abstraction

- We invoke an **A* search algorithm** over the discretized 3D airspace.
- Computes the minimum-energy feasible trajectory between every pair of locations (i, j) avoiding NFZs.
- Yields an energy matrix E_{ij} representing the exact flight cost.

Result: The physical problem reduces to routing on a complete directed graph $G = (V, A)$ where arcs A are weighted by minimum energy E_{ij} .

Formulation 1: Vehicle-Indexed MILP

Approach: Classical Capacitated Vehicle Routing Problem with Energy Constraints.

- **Variables:** $x_{ijk} \in \{0, 1\}$ tracks if drone k traverses arc (i, j) .
- **Miller-Tucker-Zemlin (MTZ) Constraints:**

$$u_{ik} + d_j \leq u_{jk} + M(1 - x_{ijk})$$

Simultaneously tracks cumulative delivered payload (u_{ik}) and eliminates disconnected subtours via strictly increasing sequences.

- **Major Drawback - Vehicle Symmetry:** Because the fleet is homogeneous, any permutation of drone indices yields the exact same objective value, causing severe branch-and-bound tree explosion.

Formulation 2: Two-Commodity Network Flow

Approach: A *vehicle-agnostic* perspective tracking two continuous "fluids".

- **Variables:** $x_{ij} \in \{0, 1\}$ (drone-independent assignment).
- **Commodity 1 (F_{ij}):** Payload flow. Balance constraint:

$$\sum_i F_{ij} - \sum_i F_{ji} = d_j, \quad \forall j \in C$$

- **Commodity 2 (Q_{ij}):** Energy flow. Accumulates energy cost E_{ij} at each node to enforce battery capacities ($Q_{ij} \leq B_{\max}$).

Key Strengths: Eliminates drone-index symmetry completely and provides a much **tighter LP Relaxation** for exact solvers compared to MTZ.

Theorem: The Drone-Based Delivery Optimization Problem is strongly NP-Hard.

Proof Outline (By Restriction):

- Consider a specialized instance of our problem where:
 - ① All Z-coordinates (altitudes) are identically 0.
 - ② The set of No-Fly Zones is empty.
 - ③ The drone battery capacity is set to infinity ($B_{\max} \rightarrow \infty$).
- The anisotropic energy model reduces to isotropic 2D Euclidean distance.
- The problem identically reduces to the classical **Capacitated Vehicle Routing Problem (CVRP)**, which is proven to be strongly NP-Hard.

Exact Method: Branch and Bound (1/2)

To mathematically prove optimal routes, we implemented a custom exact solver based on the Two-Commodity Network Flow formulation.

Algorithm Mechanics:

- **Initialization:** Uses a greedy *Nearest Neighbor* heuristic to warm-start the search and establish a strong initial **Upper Bound (UB)**.
- **Search Strategy:** Depth-First Search (DFS) over the binary assignment variables x_{ij} .
- **Bounding Oracle:** At every node in the search tree, we solve the **Linear Programming (LP) Relaxation** (relaxing $x_{ij} \in [0, 1]$) using the PuLP library to compute a strong **Lower Bound (LB)**.

Exact Method: Branch and Bound (2/2)

Branching Strategy:

- We select the "**Most Fractional Variable**" (the fractional x_{ij} closest to 0.5) to split the tree into two child nodes: $x_{ij} = 0$ (forbidden) and $x_{ij} = 1$ (mandated).

Pruning Rules:

- 1 **Infeasibility:** The LP relaxation has no valid solution under current forced arcs.
- 2 **Bound Pruning:** $LB \geq UB$. The current branch cannot possibly yield an integer solution better than the incumbent.
- 3 **Integrality Fathoming:** The LP relaxation naturally yields a pure integer solution ($x_{ij} \in \{0, 1\}$). We update the UB and prune the branch.

Because B&B scales exponentially, we developed robust metaheuristics to find high-quality Upper Bounds rapidly for larger instances.

- **Simulated Annealing (SA):**

- Uses intra-route and inter-route neighborhood operators (Swap, Insert, Reverse).
- Employs a temperature cooling schedule to probabilistically accept worse solutions, escaping local minima.

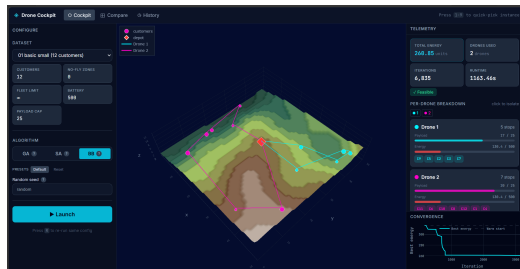
- **Genetic Algorithm (GA):**

- Maintains a population of solutions.
- **Crossover:** Uses Order Crossover (OX) to preserve feasible customer sequencing traits from parent routes.
- **Mutation:** Randomly injects diversity to explore the broader search space.

Web Platform & Implementation

Full-Stack Integration: Moving beyond theory to practical engineering.

- **Architecture:** Python FastAPI backend interacting with a React/Vite frontend.
- **3D Visualization Plane:** A central interactive 3D viewport rendering drone flight paths, NFZs, and customer nodes in real-time.
- **Run Configurator UI:** A left-panel interface to select JSON instances, choose algorithms (BB, GA, SA), and dynamically tune hyperparameters (e.g., SA cooling rate, GA population).
- **Live Streaming:** Uses Server-Sent Events (SSE) to chart energy vs. iterations live during execution.



Summary of Achievements:

- **Physical to Mathematical:** Successfully abstracted a complex 3D aerodynamic environment with NFZs into a clean graph-theoretic model using A*.
- **Rigorous Modeling:** Formulated the problem using two MILP approaches and proved its NP-Hard complexity.
- **Algorithmic Suite:** Developed an exact LP-based B&B solver alongside highly optimized SA and GA metaheuristics.
- **Software Engineering:** Packaged the computational engines into a fully integrated, interactive web-based solver platform.

Thank You! Questions?